

Spécification de la Demande d'API pour Nganda

APIs d'Authentification

0. Endpoint de Connexion

Cet endpoint permet à un utilisateur de s'authentifier auprès du système et d'obtenir un jeton d'accès.

- **Endpoint** : `POST /api/auth/login`
- **Description** : Authentifie un utilisateur avec son email et mot de passe, et renvoie un jeton d'authentification (JWT).
- **Corps de la Requête (Request Body)** :
 - **Format** : `application/json`
 - **Paramètres** :
 - `email` (Obligatoire)
 - **Type** : `string`
 - **Description** : L'adresse email de l'utilisateur.
 - **Exemple** : `"utilisateur@example.com"`
 - `password` (Obligatoire)
 - **Type** : `string`
 - **Description** : Le mot de passe de l'utilisateur.
 - **Exemple** : `"MotDePasseSecure123!"`
 -
- **Réponse JSON** :
 1. {
 2. "message": "Connexion réussie.",
 3. "token": "votre_jeton_jwt_ici...",
 4. "user": { /* détails de l'utilisateur */ }
 5. }

APIs Globales (Utilisées sur plusieurs pages)

1. Endpoint d'Initialisation

Cet endpoint charge les données essentielles à l'ouverture du dashboard pour une vue agrégée et personnalisée.

- **Endpoint** : `GET /api/dashboard/init`
 - **Description** : Récupère les données d'initialisation du dashboard, filtrées par bar(s) et période.
 - **Paramètres (Query)** :
 - `bar_id` (Optionnel)
 - **Type** : `string` ou `array de strings` (séparées par des virgules)
 - **Description** : Identifiant(s) unique(s) d'un ou plusieurs bars. Si omis, les données agrégées pour **tous les bars** seront retournées.
 - **Exemple** : `?bar_id=bar_abc` ou `?bar_id=bar_abc,bar_xyz`
 - `period` (Optionnel)
 - **Type** : `string`
 - **Description** : La période pour laquelle les données doivent être agrégées.
 - **Valeurs possibles** : `today`, `this_week`, `this_month`
 - **Valeur par défaut** : `today` (si non spécifié)
 - **Exemple** : `?period=this_month`
-

APIs pour la "Vue d'Ensemble"

2. Endpoint des KPIs Principaux

Fournit les chiffres clés et les alertes nécessaires pour la vue d'ensemble du dashboard.

- **Endpoint** : `GET /api/dashboard/overview`
 - **Description** : Récupère les indicateurs clés de performance (KPIs), les alertes et les données pour le graphique des ventes hebdomadaires.
 - **Paramètres (Query)** :
 - `period` (Obligatoire)
 - **Type** : `string`
 - **Description** : La période pour laquelle les KPIs doivent être calculés.
 - **Valeurs possibles** : `today`, `this_week`, `this_month`
 - `bar_id` (Optionnel)
 - **Type** : `string` ou `array de strings` (séparées par des virgules)
 - **Description** : Identifiant(s) unique(s) d'un ou plusieurs bars. Si omis, les données agrégées pour tous les bars seront retournées.
 - **Réponse JSON (Succès - Code HTTP 200 OK)** :
-

```

1. {
2.   "kpis": {
3.     "revenue": 1250000,
4.     "revenue_comparison": 0.05, // +5% vs precedent
5.     "average_basket": 8500,
6.     "clients_served": 147,
7.     "transactions": 120
8.   },
9.   "alerts": {
10.    "out_of_stock_count": 3,
11.    "low_stock_count": 5,
12.    "expiring_soon_count": 2,
13.    "cancelation_count": 1,
14.
15.  },
16.  "weekly_sales_chart": [
17.    { "day": "Lundi", "revenue": 800000 },
18.    { "day": "Mardi", "revenue": 1100000 },
19.    // ... autres jours
20.  ]
21.}

```

APIs pour "Analyse des Ventes"

3. Endpoint des Détails des Ventes

Fournit les données détaillées pour les graphiques et les classements de produits (tops/flops).

- **Endpoint** : `GET /api/analytics/sales`
 - **Description** : Récupère les tendances de ventes, les ventes par catégorie, les produits les plus/moins vendus et les méthodes de paiement utilisées.
 - **Paramètres (Query)** :
 - `period` (Obligatoire)
 - Type : `string`
 - Description : La période d'analyse.
 - Valeurs possibles : `today`, `this_week`, `this_month`
 - `bar_id` (Optionnel)
 - Type : `string` ou `array de strings`
 - Description : Identifiant(s) unique(s) d'un ou plusieurs bars.
 - `category` (Optionnel)
 - Type : `string`
 - Description : Filtre les données par catégorie de produit.
 - Exemple : `beers`, `softs`, `cocktails`
-

- `payment_method` (Optionnel)
 - Type : `string`
 - Description : Filtre les données par méthode de paiement.
 - Exemple : `cash`, `mobile_money`, `card`
- Réponse JSON (Succès - Code HTTP 200 OK) :


```

1. {
2.   "sales_trend_chart": [
3.     {
4.       "date": "2025-07-21",
5.       "revenue": 800000,
6.       "transactions": 80
7.     },
8.     {
9.       "date": "2025-07-22",
10.      "revenue": 1100000,
11.      "transactions": 105
12.    }
13.    // ...
14.  ],
15.  "sales_by_category_chart": [
16.    { "category": "Bières", "percentage": 0.60 },
17.    { "category": "Softs", "percentage": 0.25 },
18.    { "category": "Cocktails", "percentage": 0.15 }
19.  ],
20.  "top_products": [
21.    {
22.      "name": "Primus",
23.      "quantity_sold": 150,
24.      "revenue": 300000
25.    },
26.    // ...
27.  ],
28.  "flop_products": [
29.    {
30.      "name": "Sprite",
31.      "quantity_sold": 10,
32.      "revenue": 15000
33.    },
34.    // ...
35.  ],
36.  "peak_hours_chart": [
37.    { "hour": "18:00", "sales_volume": 15 },
38.    { "hour": "19:00", "sales_volume": 25 }
39.    // ...
40.  ],
41.  "payment_methods_chart": [
42.    { "method": "Cash", "percentage": 0.60 },
43.    { "method": "Mobile Money", "percentage": 0.40 }
44.  ]
45. }
```

APIs pour "Gestion des Stocks"

4. Endpoint de l'État des Stocks

Donne une vue d'ensemble et la liste détaillée des produits en stock, avec des options de filtrage.

- **Endpoint** : `GET /api/stocks/status`
- **Description** : Fournit un résumé de l'état des stocks et une liste détaillée des produits, y compris leur quantité et leur statut.
- **Paramètres (Query)** :
 - `bar_id` (Obligatoire)
 - Type : `string`
 - Description : Identifiant unique du bar pour lequel l'état des stocks est demandé.
 - `status_filter` (Optionnel)
 - Type : `string`
 - Description : Filtre les produits par leur statut de stock.
 - Valeurs possibles : `out_of_stock`, `low_stock`, `in_stock`
 - `search_query` (Optionnel)
 - Type : `string`
 - Description : Recherche de produits par nom ou autre critère pertinent.
 - Exemple : `coca`, `bière`
- **Réponse JSON (Succès - Code HTTP 200 OK)** :

```
1. {
2.   "summary": {
3.     "total_stock_value": 5000000,
4.     "out_of_stock_count": 3,
5.     "low_stock_count": 5
6.   },
7.   "products": [
8.     {
9.       "id": "prod_1",
10.      "name": "Turbo King",
11.      "depot_stock": 5, // en casiers
12.      "fridge_stock": 12, // en bouteilles
13.      "status": "in_stock" // in_stock, low_stock, out_of_stock
14.    },
15.    // ...
16.  ]
17.}
```

5. Endpoint de l'Historique des Mouvements de Stock

Fournit un journal des mouvements de stock pour un bar donné.

- **Endpoint** : `GET /api/stocks/history`
- **Description** : Récupère l'historique des entrées, sorties et transferts de stock.
- **Paramètres (Query)** :
 - `bar_id` (Obligatoire)
 - Type : `string`
 - Description : Identifiant unique du bar pour lequel l'historique est demandé.
 - `limit` (Optionnel)
 - Type : `integer`
 - Description : Nombre maximum d'enregistrements à retourner (pour la pagination).
 - Exemple : `20`
 - `offset` (Optionnel)
 - Type : `integer`
 - Description : Point de départ des enregistrements (pour la pagination).
 - Exemple : `0`
- **Réponse JSON (Succès - Code HTTP 200 OK)** :


```

1. [
2.   {
3.     "type": "ENTRÉE",
4.     "details": "+2 casiers de Primus",
5.     "date": "2025-07-24T10:00:00Z"
6.   },
7.   {
8.     "type": "TRANSFERT",
9.     "details": "12 bouteilles de Skol (Dépôt -> Frigo)",
10.    "date": "2025-07-24T09:00:00Z"
11.  }
12.  //...
13. ]
```

APIs pour "Performance & Équipes"

6. Endpoint de la Performance des Employés

Fournit les classements et le tableau de performance détaillé des employés.

- **Endpoint** : `GET /api/teams/performance`
- **Description** : Récupère le classement des employés par ventes et un tableau détaillé de leur performance individuelle.
- **Paramètres (Query)** :
 - `period` (Obligatoire)
 - Type : `string`

- Description : La période pour laquelle la performance est calculée.
 - Valeurs possibles : `today`, `this_week`, `this_month`
- `bar_id` (Optionnel)
 - Type : `string` ou `array de strings`
 - Description : Identifiant(s) unique(s) d'un ou plusieurs bars. Si omis, les données agrégées pour tous les bars seront retournées.
- `employee_id` (Optionnel)
 - Type : `string`
 - Description : Identifiant unique d'un employé spécifique pour obtenir ses détails de performance.

● **Réponse JSON (Succès - Code HTTP 200 OK) :**

```

1. {
2.   "leaderboard": [
3.     {
4.       "employee_id": "emp_1",
5.       "name": "John Doe",
6.       "total_sales": 350000
7.     },
8.     {
9.       "employee_id": "emp_2",
10.      "name": "Jane Smith",
11.      "total_sales": 280000
12.    }
13.    //...
14.  ],
15.  "performance_table": [
16.    {
17.      "employee_id": "emp_1",
18.      "name": "John Doe",
19.      "sales_count": 152,
20.      "revenue": 350000,
21.      "cancellation_rate": 0.015
22.    },
23.    {
24.      "employee_id": "emp_2",
25.      "name": "Jane Smith",
26.      "sales_count": 120,
27.      "revenue": 280000,
28.      "cancellation_rate": 0.05
29.    }
30.    //...
31.  ]
32.}
```

APIs pour "Analyse Comparative"

Geedoo KANDA

Software engineer | geedoo-kanda.com

geedookanda06@gmail.com | +243 972 062 655

7. Endpoint de Comparaison

Endpoint puissant qui retourne les données comparatives entre deux bars spécifiques sur différentes métriques.

- **Endpoint** : `GET /api/analytics/compare`
- **Description** : Compare les KPIs, les ventes par catégorie, les tops produits et les méthodes de paiement entre deux bars.
- **Paramètres (Query)** :
 - `period` (Obligatoire)
 - Type : `string`
 - Description : La période d'analyse pour la comparaison.
 - Valeurs possibles : `today`, `this_week`, `this_month`
 - `bar_id_1` (Obligatoire)
 - Type : `string`
 - Description : Identifiant unique du premier bar à comparer.
 - `bar_id_2` (Obligatoire)
 - Type : `string`
 - Description : Identifiant unique du second bar à comparer.
- **Réponse JSON (Succès - Code HTTP 200 OK)** :

```

1. {
2.   "kpi_comparison": [
3.     {
4.       "metric": "Chiffre d'Affaires",
5.       "bar_1_value": 1250000,
6.       "bar_2_value": 980000
7.     },
8.     {
9.       "metric": "Panier Moyen",
10.      "bar_1_value": 8500,
11.      "bar_2_value": 9100
12.     }
13.    // ...(Clients Servis, Transactions)
14.  ],
15.  "category_sales_comparison": [
16.    {
17.      "category": "Bières",
18.      "bar_1_sales": 750000,
19.      "bar_2_sales": 600000 },
20.    {
21.      "category": "Softs",
22.      "bar_1_sales": 312500,
23.      "bar_2_sales": 200000 }
24.    // ...
25.  ],
26.  "top_products_comparison": {
27.    "bar_1": [
28.      { "name": "Primus", "quantity_sold": 150 },

```



```
29.     /* ... */
30.   ],
31.   "bar_2": [
32.     { "name": "Skol", "quantity_sold": 180 },
33.     /* ... */
34.   ]
35. },
36. "payment_methods_chart": [
37.   "bar_1": [
38.     { "method": "Cash", "percentage": 0.60 },
39.     { "method": "Mobile Money", "percentage": 0.40 }
40.   ],
41.   "bar_2": [
42.     { "method": "Cash", "percentage": 0.60 },
43.     { "method": "Mobile Money", "percentage": 0.40 }
44.   ]
45. ]
46. }
```